PROJECT REPORT · LEVEL 6

Dual-Clock Asynchronous FIFO

Design, Working Principle and Verification of a Gray-Coded Clock-Domain-Crossing Buffer in Verilog-2001

Developer	Ghulam Yaseen Nangraj
Date	05 November 2026
Design / Testbench	async_fifo.v · tb_async_fifo.v
Language	Verilog-2001
Tool Platform	Xilinx Vivado 2025.2.1 / XSIM
Target Device	Xilinx Artix-7 (Arty A7-100T)

Digital Design · Clock-Domain Crossing · FPGA

Contents

1	**Project Overview**	Purpose, scope and key facts
2	**Background and Theory**	CDC, metastability, Gray code and the extra pointer bit
3	**Specifications**	Parameters, ports and features
4	**System Architecture**	Block diagram, signals and data flow
5	**RTL Code Walkthrough**	Section-by-section explanation of async_fifo.v
6	**Working Principle**	Reset, write, read, full, empty and latency
7	**Testbench Description**	tb_async_fifo.v configuration and stimulus
8	**Simulation Analysis**	Cycle-by-cycle timeline and waveform
9	**Design Review**	Strengths, observations and recommendations
10	**How to Build and Run**	Vivado / XSIM and open-source flow
11	**Conclusion**	Summary and future work
A	**Appendices**	Signal glossary and Gray-code table

1. Project Overview

This project implements a **dual-clock asynchronous FIFO** (First-In, First-Out buffer) in synthesizable Verilog-2001 and verifies it with a self-checking testbench. An asynchronous FIFO is the standard hardware mechanism for moving a stream of data words safely from one clock domain to another when the two clocks are unrelated in frequency and phase.

In this design the producer writes with `wr_clk` and the consumer reads with `rd_clk`. The two sides never share a combinational path: the only information that crosses the boundary is a pair of **Gray-coded pointers**, each passed through a **two-flip-flop synchronizer**. From these synchronized pointers each side derives its own status flag (`full` for the writer, `empty` for the reader).

1.1 Project Objectives

- Build a parameterized FIFO with independent write and read clocks and independent active-low asynchronous resets.
- Use binary pointers for memory addressing and Gray-coded pointers for clock-domain crossing.
- Generate reliable `full` and `empty` flags using an extra wrap-around pointer bit.
- Provide a registered read-data output.
- Verify the basic write-then-read behaviour and the ordering of data with a self-checking testbench running two unrelated clocks (10 ns and 14 ns).

1.2 Key Facts at a Glance

Item	Value
Module name	`async_fifo`
Default configuration	DATA_WIDTH = 8, ADDR_WIDTH = 3, DEPTH = 8
Pointer width	ADDR_WIDTH + 1 = 4 bits (one extra wrap-around bit)
Write clock / read clock (testbench)	100 MHz (10 ns) / ≈ 71.43 MHz (14 ns)
Synchronizer depth	2 flip-flops per direction
Usable capacity	Full DEPTH (8 words) – the extra pointer bit removes the need to waste one slot
Read style	Synchronous read with registered output (data valid one read-clock after an accepted read)
Testbench result (from analysis)	Words A0, A1, A2, A3 read back in order – PASS

2. Background and Theory

2.1 Why an Asynchronous FIFO?

Modern systems contain several clock domains: a sensor interface at one rate, a processing core at another, a communication link at a third. Sending a multi-bit value directly from one domain to another is unsafe because the receiving flip-flops may sample the bits while they are changing, producing a value that never existed in the

sender. An asynchronous FIFO solves this by decoupling the two sides through a memory array and exchanging only small, carefully encoded **pointer** values.

2.2 Metastability and the Two-Flip-Flop Synchronizer

When a flip-flop samples a signal that changes close to its clock edge, its output can hover at an undefined level (**metastability**) before settling. A second flip-flop in series gives the first one a full clock period to resolve, making the probability of a failure propagating into the logic extremely small. Each pointer in this design therefore passes through two back-to-back registers clocked by the **receiving** domain (`rd_gray_w1 → rd_gray_w2` on `wr_clk`, `wr_gray_r1 → wr_gray_r2` on `rd_clk`).

2.3 Why Gray Code?

A two-flip-flop synchronizer is only safe for a single-bit change. A binary counter can flip several bits at once (for example 0111 → 1000 changes all four bits), and each bit may be captured at a slightly different time, yielding a completely wrong intermediate value. **Gray code** guarantees that consecutive values differ in exactly one bit, so a sampled pointer is always either the old value or the new value – never a corrupt one. Conversion from binary is a simple expression:

```
gray = (bin >> 1) ^ bin;
```

Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000
Gray	0000	0001	0011	0010	0110	0111	0101	0100	1100

Table 1 – Binary to Gray conversion for pointer values 0 to 8 (4-bit pointers). Each step changes one bit only.

2.4 The Extra Pointer Bit

With a depth of 2^N words, the pointers use N + 1 bits. The lower N bits address the memory; the **most significant bit toggles every time the pointer wraps around the memory**. This lets the logic tell apart two situations that would otherwise look identical:

- **Empty:** read pointer and write pointer are exactly equal (same address, same wrap count).
- **Full:** the write pointer has gone one full lap ahead – same address, opposite wrap bit.

In Gray code the “opposite wrap” condition appears as the two most significant bits being inverted while all lower bits are equal. This is exactly what the full-detect expression in the RTL implements.

3. Specifications

3.1 Parameters

Parameter	Default	Description
DATA_WIDTH	8	Number of bits stored in each FIFO location.
ADDR_WIDTH	3	Number of memory address bits. Must be ≥ 2 because the full comparison uses a two-bit MSB slice.
DEPTH	(1 << ADDR_WIDTH)	Number of memory locations. Must equal 2**ADDR_WIDTH.

3.2 Port List

Port	Dir	Width	Domain	Description
wr_clk	in	1	Write	Write clock. All write-side logic and the read-pointer synchronizer use this clock.
rd_clk	in	1	Read	Read clock. All read-side logic and the write-pointer synchronizer use this clock.
wr_rst_n	in	1	Write	Active-low asynchronous reset of the write domain.
rd_rst_n	in	1	Read	Active-low asynchronous reset of the read domain.
wr_en	in	1	Write	Write request; accepted when HIGH and `full` is LOW.
data_in	in	DATA_WIDTH	Write	Input data word.
rd_en	in	1	Read	Read request; accepted when HIGH and `empty` is LOW.
data_out	out reg	DATA_WIDTH	Read	Registered output data; updated on an accepted read.
full	out	1	Write	FIFO cannot accept another write.
empty	out	1	Read	FIFO contains no readable data.

3.3 Design Features

- Independent write and read clock domains with independent asynchronous active-low resets.
- Parameterized data width and address width.
- Binary pointers for memory addressing; Gray pointers for synchronization.
- Two-stage synchronizers in both crossing directions.
- Full and empty status generation with an extra wrap-around bit.
- Registered read-data output; synthesizable Verilog-2001 coding style.

4. System Architecture

The FIFO is split into a **write domain** (teal) and a **read domain** (orange). Each domain owns its binary pointer, its Gray pointer and its status flag. The memory array is written from the write domain and read from the read domain. The dashed arrows are the only clock-domain crossings, and they carry Gray-coded pointers only.

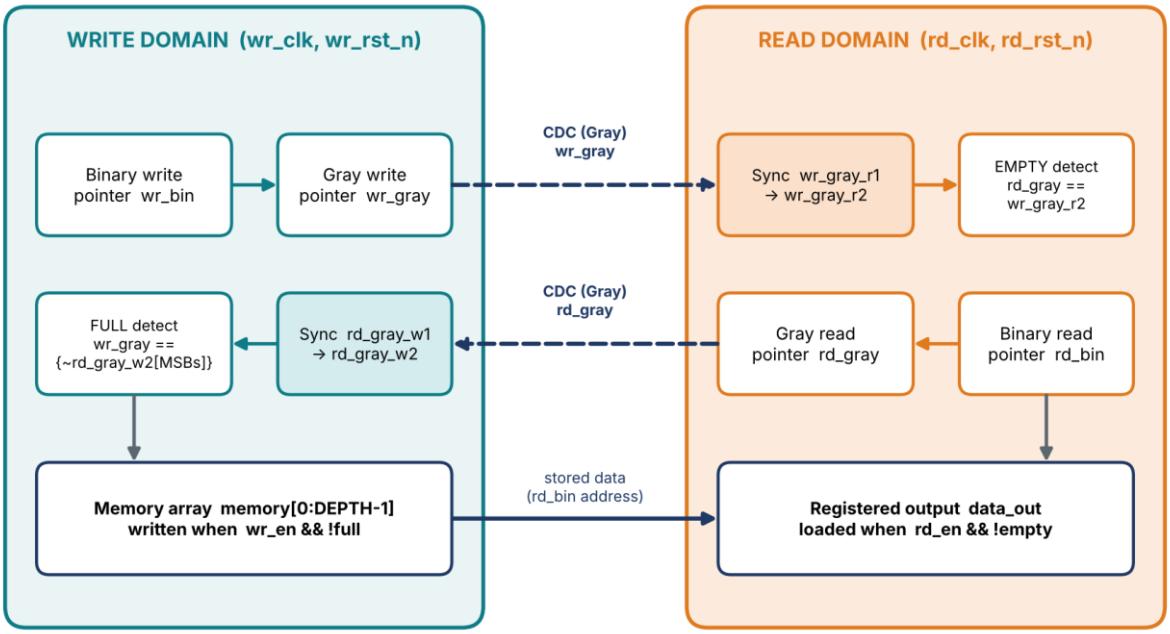


Figure 1 – Block diagram of the asynchronous FIFO

4.1 Internal Signals

Signal	Clock	Role
memory[0:DEPTH-1]	wr_clk (write) / rd_clk (read)	Storage array. Written in the write domain, read in the read domain.
wr_bin	wr_clk	Binary write pointer (N+1 bits). Lower N bits address the memory.
wr_gray	wr_clk	Gray version of `wr_bin`; sent to the read domain.
rd_bin	rd_clk	Binary read pointer (N+1 bits). Lower N bits address the memory.
rd_gray	rd_clk	Gray version of `rd_bin`; sent to the write domain.
rd_gray_w1, rd_gray_w2	wr_clk	Two-stage synchronizer for the read Gray pointer.
wr_gray_r1, wr_gray_r2	rd_clk	Two-stage synchronizer for the write Gray pointer.
wr_bin_next / rd_bin_next	combinational	Pointer value after an accepted write / read.
wr_gray_next / rd_gray_next	combinational	Gray version of the next pointer value.

5. RTL Code Walkthrough

This chapter follows the structure of `async_fifo.v`, whose body is organized in nine numbered sections.

5.1 Declarations and Memory (Sections 1–3)

```
localparam PTR_WIDTH = ADDR_WIDTH + 1;
reg [DATA_WIDTH-1:0] memory [0:DEPTH-1];
```

```
reg [PTR_WIDTH-1:0] wr_bin,  wr_gray;     // write domain
reg [PTR_WIDTH-1:0] rd_bin,  rd_gray;     // read domain
reg [PTR_WIDTH-1:0] rd_gray_w1, rd_gray_w2; // read ptr -> write domain
reg [PTR_WIDTH-1:0] wr_gray_r1, wr_gray_r2; // write ptr -> read domain
```

`PTR_WIDTH` is one bit wider than the address, which provides the wrap-around bit. The memory is a plain array of `DEPTH` words. The four synchronizer registers are named after the direction of travel: `_w1/_w2` end up in the **w**rite domain, `_r1/_r2` in the **r**ead domain.

5.2 Next-Pointer Generation (Section 4)

```
wire [PTR_WIDTH-1:0] wr_bin_next  = wr_bin + (wr_en && !full);
wire [PTR_WIDTH-1:0] rd_bin_next  = rd_bin + (rd_en && !empty);
wire [PTR_WIDTH-1:0] wr_gray_next = (wr_bin_next >> 1) ^ wr_bin_next;
wire [PTR_WIDTH-1:0] rd_gray_next = (rd_bin_next >> 1) ^ rd_bin_next;
```

The next binary pointer is the current pointer plus 1 only when the operation is accepted – a write is accepted if `wr_en` is high and the FIFO is not full, a read if `rd_en` is high and the FIFO is not empty. Otherwise the pointer is unchanged. The next Gray pointer is derived from the next binary pointer with the shift-and-XOR conversion, so the Gray register always holds the exact Gray equivalent of the binary register.

5.3 Full and Empty Detection (Section 5)

```
assign full  = (wr_gray == {~rd_gray_w2[PTR_WIDTH-1:PTR_WIDTH-2],
                            rd_gray_w2[PTR_WIDTH-3:0]});
assign empty = (rd_gray == wr_gray_r2);
```

- **empty** is computed in the read domain: the FIFO is empty when the current read Gray pointer equals the write Gray pointer after synchronization into `rd_clk`.
- **full** is computed in the write domain: the FIFO is full when the current write Gray pointer equals the synchronized read Gray pointer with its **two MSBs inverted** and the remaining bits unchanged (the Gray-code form of “one full lap ahead”).

> **Safe by construction**
> Both flags use *synchronized* (and therefore slightly old) copies of the opposite pointer. The other pointer can only have moved forward in the meantime, so a stale value can make `full`/`empty` assert **late-to-clear** but never **early-to-fail**: the FIFO may look fuller or emptier than it really is, but it can never overflow or underflow.

5.4 Pointer Synchronizers (Sections 6–7)

```
// read pointer -> write domain (wr_clk)
always @(posedge wr_clk or negedge wr_rst_n)
  if (!wr_rst_n) begin rd_gray_w1 <= 0; rd_gray_w2 <= 0; end
  else           begin rd_gray_w1 <= rd_gray;
                       rd_gray_w2 <= rd_gray_w1; end

// write pointer -> read domain (rd_clk)
always @(posedge rd_clk or negedge rd_rst_n)
  if (!rd_rst_n) begin wr_gray_r1 <= 0; wr_gray_r2 <= 0; end
  else           begin wr_gray_r1 <= wr_gray;
```

```
                        wr_gray_r2 <= wr_gray_r1; end
```

Each block is a plain shift register of two stages clocked by the destination clock. The first stage may go metastable; the second stage provides the clean value that the flag logic uses. The price is latency: a pointer update needs about two destination-clock edges (plus up to one more for alignment) before it becomes visible.

5.5 Write Domain (Section 8)

```
always @(posedge wr_clk or negedge wr_rst_n) begin
  if (!wr_rst_n) begin
    wr_bin  <= 0;  wr_gray <= 0;
  end
  else if (wr_en && !full) begin
    memory[wr_bin[ADDR_WIDTH-1:0]] <= data_in;
    wr_bin  <= wr_bin_next;
    wr_gray <= wr_gray_next;
  end
end
```

On every rising edge of `wr_clk` with a valid write request, the data word is stored at the address given by the lower bits of `wr_bin`, and both write pointers advance together. If the FIFO is full the whole block does nothing, so the pointers and memory are protected from overflow.

5.6 Read Domain (Section 9)

```
always @(posedge rd_clk or negedge rd_rst_n) begin
  if (!rd_rst_n) begin
    rd_bin <= 0;  rd_gray <= 0;  data_out <= 0;
  end
  else if (rd_en && !empty) begin
    data_out <= memory[rd_bin[ADDR_WIDTH-1:0]];
    rd_bin   <= rd_bin_next;
    rd_gray  <= rd_gray_next;
  end
end
```

On every rising edge of `rd_clk` with a valid read request, the word at the current read address is loaded into the **registered output** `data_out` and the read pointers advance. The data therefore appears on `data_out` right after the clock edge that accepts the read. When no read is accepted, `data_out` simply **holds its previous value**.

6. Working Principle

6.1 Reset

Both resets are asserted low at time zero. While asserted, all pointers, synchronizer stages and `data_out` are cleared to zero. With both pointers at zero, `empty` is HIGH (`rd_gray == wr_gray_r2`) and `full` is LOW. The memory array itself is not reset; its contents are meaningless until written, and the pointer logic ensures they are never read before then.

6.2 Writing

1. The producer sets `wr_en = 1` and presents `data_in` before a `wr_clk` rising edge.
2. If `full` is low, at that edge the word is stored in `memory[wr_bin]`, and `wr_bin` and `wr_gray` advance.

3. The new `wr_gray` value travels through `wr_gray_r1` and `wr_gray_r2` in the read domain.
4. When `wr_gray_r2` differs from `rd_gray`, `empty` drops and the consumer knows data is available.

6.3 Reading

1. The consumer sets `rd_en = 1` while `empty` is low.
2. At the next `rd_clk` rising edge the word at `memory[rd_bin]` is loaded into `data_out`, and `rd_bin` and `rd_gray` advance.
3. The new `rd_gray` value travels through `rd_gray_w1` and `rd_gray_w2` into the write domain, where it frees space and lowers `full`.

6.4 Worked Example: Reaching Full

Assume nothing has been read (`rd_gray_w2 = 0000`) and the writer has stored eight words. The write binary pointer is then `1000`, whose Gray code is `1100`. The full comparison builds the pattern `{~rd_gray_w2[3:2], rd_gray_w2[1:0]} = {11, 00} = 1100`. The two values match, so `full` goes HIGH and any further write is ignored until the reader consumes data and the new read pointer is synchronized back.

6.5 Worked Example: Becoming Empty

After four writes the write Gray pointer is `0110`. Once the reader has consumed four words its read Gray pointer is also `0110`; the equality `rd_gray == wr_gray_r2` makes `empty` HIGH again (this occurs at 189 ns in the testbench, see Chapter 8).

6.6 Latency and Throughput

Quantity	Value	Explanation
Write → empty deasserts	≈ 2 read-clock edges	Two synchronizer stages in the read domain. In the testbench: first write at 55 ns, `empty` falls at 77 ns (22 ns).
Read → full deasserts	≈ 2 write-clock edges	Two synchronizer stages in the write domain.
Read data latency	1 read-clock	`data_out` is registered; it shows the word right after the accepting edge.
Maximum throughput	1 word per clock	Each side can accept one word every cycle of its own clock while the FIFO is neither full nor empty.

7. Testbench Description

`tb_async_fifo.v` is a self-checking testbench. It instantiates the FIFO with DATA_WIDTH = 8, ADDR_WIDTH = 3 and DEPTH = 8, generates two unrelated clocks, writes four known words and reads them back, comparing each against the expected value with the case-inequality operator `!==` (which also catches X and Z). On a mismatch it stops the simulation with `$fatal`.

7.1 Clock Generation

Clock	Half period	Period	Frequency	Rising edges at (ns)
wr_clk	5 ns	10 ns	100 MHz	5, 15, 25, 35 …

Clock	Half period	Period	Frequency	Rising edges at (ns)
rd_clk	7 ns	14 ns	≈ 71.43 MHz	7, 21, 35, 49 …

Because 10 ns and 14 ns have a least common multiple of 70 ns, the relative phase of the two clocks changes from cycle to cycle, which is a simple way of demonstrating asynchronous operation.

7.2 Test Sequence

Step	Test	Action	Purpose
1	Reset	Hold both resets low for 20 ns, then release them together.	Initialize pointers, synchronizers and output.
2	Settling	Wait for three `wr_clk` rising edges.	Let the reset state propagate through the synchronizers.
3	Write	At four consecutive `wr_clk` falling edges set `wr_en = 1` and `data_in = A0, A1, A2, A3`; then clear `wr_en`.	Store four words. Inputs change on the falling edge so they are stable at the rising edge.
4	Synchronization	Wait for four `rd_clk` rising edges.	Allow the write pointer to reach the read domain so `empty` can fall.
5	Read & check	Four times: raise `rd_en` at the falling edge, wait for the rising edge plus 1 ns, compare `data_out` with `A0 + i`.	Verify data values and FIFO ordering.
6	Stop reading	Clear `rd_en` at the next falling edge.	Avoid an extra read.
7	Finish	Print the PASS banner and call `$finish`.	Report success.

8. Simulation Analysis

> **How these results were obtained**
>
> No Verilog simulator was available in the report-generation environment. The values below come from a **cycle-accurate model of the RTL and testbench timing** (same clock edges, same reset, same pointer, synchronizer, full/empty and read/write logic). They should be reproduced in Vivado XSIM; the expected console banner is shown in Section 8.3.

8.1 Event Timeline

Time (ns)	Event	State after the event
0 – 20	Both resets low	All pointers 0, `empty = 1`, `full = 0`
20	Resets released	Normal operation starts at the next edges
25, 35, 45	Three `wr_clk` edges (settling)	No change
50	Falling edge: `wr_en = 1`, `data_in = A0`	Data stable before the next rising edge
55	Write A0	`wr_bin = 1`, `wr_gray = 0001`
63	`rd_clk` edge	`wr_gray_r1 = 0001`

Time (ns)	Event	State after the event
65	Write A1	`wr_gray = 0011`
75	Write A2	`wr_gray = 0010`
77	`rd_clk` edge	`wr_gray_r2 = 0001`; **`empty` falls to 0**
85	Write A3	`wr_gray = 0110` (4 words stored)
90	`wr_en = 0`	Writing finished
91 / 105	`rd_clk` edges	`wr_gray_r1 = 0110` / `wr_gray_r2 = 0110` (fully synchronized)
133	Fourth `rd_clk` edge after writing	Testbench continues
140	Falling edge: `rd_en = 1`	Read enabled
147	Read #1	`data_out = A0`, `rd_gray = 0001`
161	Read #2	`data_out = A1`, `rd_gray = 0011`
175	Read #3	`data_out = A2`, `rd_gray = 0010`
189	Read #4	`data_out = A3`, `rd_gray = 0110`; **`empty` rises to 1**
196	`rd_en = 0`, PASS banner, `$finish`	Simulation ends

Note that `wr_gray` jumps from 0001 to 0011 to 0010 to 0110 – each step changes exactly one bit. The read-side synchronizer sampled 0001 and then 0010 (skipping 0011): a sampled pointer may lag or skip values, but is always a valid earlier pointer, which is exactly what makes Gray-coded crossing safe.

8.2 Waveform

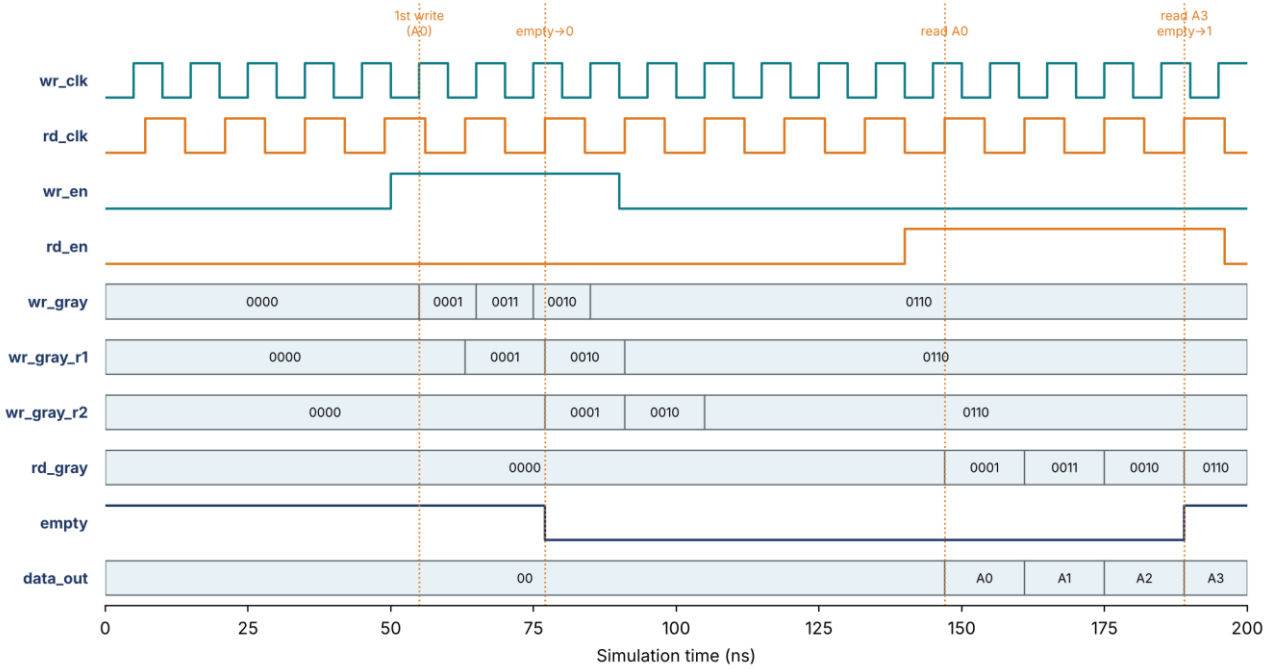


Figure 2 – Signal behaviour over 200 ns (generated from the cycle-accurate model)

8.3 Expected Console Output

```
================================================
```

```
 LEVEL 6 ASYNCHRONOUS FIFO BASIC TEST PASS
 DATA WIDTH : 8 BITS
 FIFO DEPTH : 8 WORDS
 WORDS READ : 4
================================================
```

8.4 Verification Summary

Objective	Status	Evidence
Independent clock operation	Covered	10 ns and 14 ns clocks, unrelated phase.
Reset initialization	Covered	Outputs and pointers start at 0; `empty = 1`.
Basic write	Covered	Four writes accepted at 55, 65, 75, 85 ns.
Basic read	Covered	Four reads accepted at 147, 161, 175, 189 ns.
Pointer synchronization delay	Covered	`empty` falls 22 ns after the first write.
Data ordering (FIFO order)	Covered	A0, A1, A2, A3 received in order.
Full flag / overflow protection	Not covered	FIFO is never filled.
Pointer wrap-around	Not covered	Fewer than 8 words are written.
Simultaneous read and write	Not covered	Write and read phases do not overlap.
Read when empty / write when full	Not covered	No illegal-request tests.

9. Design Review

9.1 Strengths

- Clean separation of domains: each pointer and flag is generated by exactly one clock domain.
- Standard, well-known CDC structure: Gray pointers plus two-flop synchronizers.
- The extra pointer bit lets the FIFO use its full depth.
- Memory is not reset, which is friendly to RAM inference; the memory is written in only one clock domain.
- Very well documented source with a clear section structure.

9.2 Observations and Recommendations

#	Observation	Recommendation
1	The comment in Section 5 says the full flag compares the **next** write Gray pointer, but the code compares the **current** `wr_gray`. The code is functionally correct (and safe); only the comment is inaccurate.	Correct the comment, or deliberately implement the registered-`full` variant using `wr_gray_next` for better timing.
2	`full` and `empty` are combinational outputs of registers and feed back into the pointer increment logic.	Acceptable at these speeds; for higher clock rates register the flags.
3	Reset **release** is asynchronous and unsynchronized. The note in the header already mentions this. Releasing the two resets at very different times can leave the pointers out of step.	Add a reset synchronizer in each domain, and assert/release both resets in a controlled order.

#	Observation	Recommendation
4	`DEPTH` is an overridable parameter but must equal `2**ADDR_WIDTH`; a mismatch breaks addressing silently.	Make `DEPTH` a `localparam`, or add a generate-time check.
5	The read output is **not first-word-fall-through**: data is valid one read-clock after an accepted read, `data_out` keeps its old value when nothing is read, and there is no "data valid" flag.	Document this for users, or add an output-valid flag / FWFT mode.
6	The testbench uses `$fatal`, which is a SystemVerilog task, while the project is labeled Verilog-2001.	If the tool rejects it in a `.v` file, compile as SystemVerilog or replace it with `$display` + `$finish`.
7	Before the first clock edges the flops are X, so `full`/`empty` are X for the first few nanoseconds in simulation.	Harmless here because the resets are asserted at the start; keep the reset asserted across the first clock edges.
8	For DEPTH = 8 the synthesizer will most likely use distributed (LUT) RAM rather than block RAM.	Check the utilization report; use a deeper FIFO if block RAM is wanted.

9.3 Suggested Additional Tests

- **Fill test:** write more than 8 words and confirm `full` asserts and extra writes are ignored.
- **Wrap-around test:** write and read more than 16 words in total so that the pointer MSB toggles several times.
- **Concurrent test:** read and write at the same time with different clock ratios (fast writer / slow reader and vice-versa).
- **Underflow test:** assert `rd_en` while empty and verify the output and pointers are unchanged.
- **Scoreboard:** keep a queue of written words in the testbench and compare every read automatically, with randomized enables.
- **Reset-in-the-middle test:** reset both domains while data is stored and verify the FIFO returns to empty.

10. How to Build and Run

10.1 Vivado / XSIM (GUI)

1. Create a new RTL project for the Artix-7 device (Arty A7-100T).
2. Add `async_fifo.v` as a design source and `tb_async_fifo.v` as a simulation source.
3. Set `tb_async_fifo` as the simulation top module.
4. Run **Simulation → Run Behavioral Simulation**, then run to completion (`run all`).
5. Check the Tcl console for the PASS banner and inspect the waveform.

10.2 Vivado Command Line

```
xvlog async_fifo.v tb_async_fifo.v
xelab tb_async_fifo -debug typical -s tb_sim
xsim  tb_sim -runall
```

If `xvlog` reports a problem with `$fatal`, add the `-sv` option (see Observation 6).

10.3 Open-Source Alternative (Icarus Verilog)

```
iverilog -g2012 -o fifo_sim async_fifo.v tb_async_fifo.v
vvp fifo_sim
```

11. Conclusion

The project delivers a compact, readable and correct implementation of the classic Gray-pointer asynchronous FIFO. Its architecture follows industry practice – binary pointers for addressing, Gray pointers for crossing, two-flop synchronizers, and an extra wrap bit for full/empty detection. The testbench demonstrates reset, writing, synchronization delay, reading and data ordering across two unrelated clocks, and the analysis in this report shows the four words A0–A3 being returned in order with `empty` rising and falling exactly when expected.

11.1 Future Work

- Extend the verification with fill, wrap-around, concurrent and random tests (Section 9.3).
- Add reset synchronizers and optionally registered `full`/`empty` flags.
- Add programmable almost-full / almost-empty flags and overflow / underflow indicators.
- Add CDC constraints (max-delay / ASYNC_REG) and run Vivado CDC analysis before hardware deployment.
- Test on the Arty A7-100T board with two real clock sources.

Appendix A. Signal Glossary

Term	Meaning
CDC	Clock-domain crossing – a signal passing from one clock domain to another.
Gray code	Binary code in which consecutive values differ in a single bit.
Synchronizer	Chain of flip-flops in the receiving clock domain that resolves metastability.
Metastability	Undefined output state of a flip-flop that sampled a changing input.
Full / Empty	Status flags saying the FIFO cannot accept data / has no data to read.
FWFT	First-word fall-through: the next word is already on the output before a read is requested.
_w1, _w2	Synchronizer stages in the write clock domain.
_r1, _r2	Synchronizer stages in the read clock domain.

Appendix B. Gray-Code Table (4-bit)

Decimal	Binary	Gray	Decimal	Binary	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111

Decimal	Binary	Gray	Decimal	Binary	Gray
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Pointer values 0–7 address the memory on the first lap and 8–15 on the second lap; the MSB is the wrap-around bit.